TODO LIST INTERACTIVE EN PYTHON

De la liste papier à l'application de gestion de tâches

C Premiers Pas - vCODE #6

Les bases solides d'aujourd'hui sont les succès de demain

Table des matières

- Introduction : Gérer tes tâches comme un pro
- Prérequis : Ce que tu dois savoir
- PARTIE 1 : Les listes avancées (45min)
  - Étape 1 : Créer et afficher une liste
  - Étape 2 : Ajouter des éléments (append)
  - Étape 3 : Supprimer des éléments (remove, pop)
  - Étape 4 : Modifier des éléments
- PARTIE 2 : Les fichiers (45min)
  - Étape 5 : Écrire dans un fichier
  - Étape 6 : Lire depuis un fichier
  - Étape 7 : Le format JSON
- PARTIE 3 : TODO List complète (30min)
  - Étape 8 : Version console
  - Étape 9 : Sauvegarde automatique
  - Étape 10 : Version finale
- Aller plus loin
- Récapitulatif

- Conclusion

Introduction : Gérer tes tâches comme un pro

Le problème concret

Tu notes tes tâches sur du papier ? Dans ton téléphone ? Tu les oublies souvent ?

Problèmes classiques :

- ⇨ Papier perdu ou froissé
- ⇨ Tâches oubliées
- ⇨ Pas de suivi de progression
- ⇨ Impossible de partager avec d'autres

Et si tu créais TON PROPRE gestionnaire de tâches ?

La solution : Une TODO List Python

Aujourd'hui, tu vas créer une application qui :

```
╔═══════════════════════════════╗
║   TODO LIST PYTHON            ║
╟═══════════════════════════════╢
║                               ║
║  ✓ Ajouter des tâches         ║
║  ✓ Marquer comme terminé      ║
║  ✓ Supprimer des tâches       ║
║  ✓ Sauvegarder automatiquement ║
║  ✓ Charger au démarrage       ║
║  ✓ Données persistantes (JSON) ║
║                               ║
╚═══════════════════════════════╝
```

Ce que tu vas apprendre

Compétence	Ce que ça t'apporte
Listes avancées	Manipuler des collections de données
Méthodes de liste	append, remove, pop, insert
Fichiers texte	Sauvegarder et charger des données
Format JSON	Stocker des structures complexes
CRUD	Create, Read, Update, Delete
Persistance	Données qui survivent après fermeture

Résultat final attendu

À la fin de vCODE #6, tu auras :

✓ Une TODO list fonctionnelle en console
✓ Sauvegarde automatique dans un fichier JSON
✓ Chargement automatique au démarrage
✓ Gestion complète des tâches (CRUD)
✓ Code propre et réutilisable

Prérequis : Ce que tu dois savoir

Concepts Python de base

Concept	Exemple	Niveau requis
Variables	`tache = "Coder"`	✓ Obligatoire
Listes	`taches = []`	✓ Obligatoire
Boucles for	`for tache in taches:`	✓ Obligatoire
Fonctions	`def ajouter():`	✓ Obligatoire
Conditions	`if choix == "1":`	✓ Obligatoire

Si tu as fait vCODE #3 ou #4, tu es prêt !

Nouveaux concepts d'aujourd'hui

1. Les listes (rappel et approfondissement)

```
# Créer une liste
mes_taches = []

# Ajouter un élément
mes_taches.append("Apprendre Python")

# Afficher
print(mes_taches)  # ['Apprendre Python']
```

2. Les fichiers

```
# Écrire dans un fichier
with open("taches.txt", "w") as fichier:
    fichier.write("Ma première tâche")

# Lire depuis un fichier
with open("taches.txt", "r") as fichier:
    contenu = fichier.read()
```

3. Le format JSON

```
import json

# Sauvegarder
data = {"taches": ["Tâche 1", "Tâche 2"]}
with open("data.json", "w") as f:
    json.dump(data, f)

# Charger
with open("data.json", "r") as f:
    data = json.load(f)
```

PARTIE 1 : Les listes avancées (45min)

ÉTAPE 1 : Créer et afficher une liste

Objectif

Créer une liste de tâches et l'afficher proprement.

Le code (Version simple)

```
# TODO List v1.0 - Liste basique

# Créer une liste de tâches
taches = [
    "Apprendre Python",
    "Faire les courses",
    "Réviser le code"
]

# Afficher la liste
print("=== MES TÂCHES ===")
for tache in taches:
    print(f"- {tache}")
```

Explication ligne par ligne

Lignes 4-8 :

```
taches = [
    "Apprendre Python",
    "Faire les courses",
    "Réviser le code"
]
```

→ **Création d'une liste avec 3 éléments.**

Point clé : Les éléments sont séparés par des virgules.

Lignes 11-13 :

```
print("=== MES TÂCHES ===")
for tache in taches:
    print(f"- {tache}")
```

→ **Boucle for pour afficher chaque élément.**

Ce qui se passe :

- `for tache in taches:` → Pour chaque élément de la liste
- `print(f"- {tache}")` → Affiche avec un tiret devant

Affichage numéroté

```
# Afficher avec des numéros
print("=== MES TÂCHES ===")
for i, tache in enumerate(taches, 1):
    print(f"{i}. {tache}")
```

enumerate(taches, 1) :

- Donne un numéro à chaque élément
- `1` = commence la numérotation à 1 (au lieu de 0)

Résultat attendu

```
=== MES TÂCHES ===
- Apprendre Python
- Faire les courses
- Réviser le code
```

Avec numéros :

```
=== MES TÂCHES ===
1. Apprendre Python
2. Faire les courses
3. Réviser le code
```

ÉTAPE 2 : Ajouter des éléments (append)

Objectif

Permettre à l'utilisateur d'ajouter de nouvelles tâches.

Le code (Version interactive)

```python
# TODO List v2.0 - Ajout de tâches

taches = []

def afficher_taches():
    """Affiche toutes les tâches"""
    if not taches:
        print("\n✓ Aucune tâche pour le moment.")
    else:
        print("\n=== MES TÂCHES ===")
        for i, tache in enumerate(taches, 1):
            print(f"{i}. {tache}")

def ajouter_tache():
    """Ajoute une nouvelle tâche"""
    nouvelle_tache = input("\nNouvelle tâche : ")
    taches.append(nouvelle_tache)
    print(f"✓ Tâche '{nouvelle_tache}' ajoutée !")

# Programme principal
while True:
    print("\n--- MENU ---")
    print("1. Afficher les tâches")
    print("2. Ajouter une tâche")
    print("3. Quitter")

    choix = input("\nChoix : ")

    if choix == "1":
        afficher_taches()
    elif choix == "2":
        ajouter_tache()
    elif choix == "3":
        print("\nÀ bientôt !")
        break
    else:
        print("\n☒ Choix invalide !")
```

Explication ligne par ligne

Ligne 7 :

```
if not taches:
```

→ **Vérifier si la liste est vide.**

Ce qui se passe :

- `not taches` = True si la liste est vide
- Équivalent à `if len(taches) == 0:`

Ligne 17 :

```
taches.append(nouvelle_tache)
```

→ **Méthode .append() !**

Ce qui se passe :

- Ajoute l'élément **à la fin** de la liste
- La liste est modifiée directement

Les méthodes d'ajout

Méthode	Utilité	Exemple
`.append(x)`	Ajoute x à la fin	`liste.append("Nouveau")`
`.insert(i, x)`	Ajoute x à la position i	`liste.insert(0, "Premier")`
`.extend(liste2)`	Ajoute tous les éléments de liste2	`liste.extend([1, 2, 3])`

Exemples :

```python
taches = ["Tâche 1", "Tâche 2"]

# append : ajoute à la fin
taches.append("Tâche 3")
# Résultat : ["Tâche 1", "Tâche 2", "Tâche 3"]

# insert : ajoute à une position précise
taches.insert(0, "URGENT")
# Résultat : ["URGENT", "Tâche 1", "Tâche 2", "Tâche 3"]

# extend : ajoute plusieurs éléments
taches.extend(["Tâche 4", "Tâche 5"])
# Résultat : ["URGENT", "Tâche 1", "Tâche 2", "Tâche 3", "Tâche 4", "Tâche 5"]
```

Résultat attendu

```
--- MENU ---
1. Afficher les tâches
2. Ajouter une tâche
3. Quitter

Choix : 2

Nouvelle tâche : Apprendre Python
✓ Tâche 'Apprendre Python' ajoutée !

--- MENU ---
1. Afficher les tâches
2. Ajouter une tâche
3. Quitter

Choix : 1

=== MES TÂCHES ===
1. Apprendre Python
```

ÉTAPE 3 : Supprimer des éléments (remove, pop)

Objectif

Permettre de supprimer des tâches de la liste.

Le code (Version avec suppression)

```python
# TODO List v3.0 - Suppression de tâches

taches = []

def afficher_taches():
    if not taches:
        print("\n✓ Aucune tâche.")
    else:
        print("\n=== MES TÂCHES ===")
        for i, tache in enumerate(taches, 1):
            print(f"{i}. {tache}")

def ajouter_tache():
    nouvelle_tache = input("\nNouvelle tâche : ")
    taches.append(nouvelle_tache)
    print(f"✓ '{nouvelle_tache}' ajoutée !")

def supprimer_tache():
    """Supprime une tâche par son numéro"""
    if not taches:
        print("\n☒ Aucune tâche à supprimer !")
        return

    afficher_taches()

    try:
        numero = int(input("\nNuméro de la tâche à supprimer : "))

        if 1 <= numero <= len(taches):
            tache_supprimee = taches.pop(numero - 1)
            print(f"✓ Tâche '{tache_supprimee}' supprimée !")
        else:
            print("\n☒ Numéro invalide !")
    except ValueError:
        print("\n☒ Entre un numéro valide !")

# Programme principal
while True:
    print("\n--- MENU ---")
    print("1. Afficher les tâches")
    print("2. Ajouter une tâche")
```

```
    print("3. Supprimer une tâche")
    print("4. Quitter")

    choix = input("\nChoix : ")

    if choix == "1":
        afficher_taches()
    elif choix == "2":
        ajouter_tache()
    elif choix == "3":
        supprimer_tache()
    elif choix == "4":
        print("\nÀ bientôt !")
        break
    else:
        print("\n☒ Choix invalide !")
```

Explication ligne par ligne

Ligne 30 :

```
tache_supprimee = taches.pop(numero - 1)
```

→ **Méthode .pop() !**

Ce qui se passe :

- `.pop(index)` supprime l'élément à la position `index`
- **ET retourne** l'élément supprimé
- `numero - 1` car les listes commencent à 0, mais l'utilisateur voit 1, 2, 3...

Les méthodes de suppression

Méthode	Utilité	Retourne	Exemple
`.remove(x)`	Supprime la 1ère occurrence de x	Rien	`liste.remove("Tâche 1")`
`.pop(i)`	Supprime l'élément à l'index i	L'élément supprimé	`element = liste.pop(0)`

Méthode	Utilité	Retourne	Exemple
`.pop()`	Supprime le dernier élément	L'élément supprimé	`dernier = liste.pop()`
`.clear()`	Vide toute la liste	Rien	`liste.clear()`

Exemples :

```
taches = ["Urgent", "Moyen", "Plus tard", "Optionnel"]

# remove : supprime par valeur
taches.remove("Moyen")
# Résultat : ["Urgent", "Plus tard", "Optionnel"]

# pop avec index : supprime et retourne
supprimee = taches.pop(0)  # supprimee = "Urgent"
# Résultat : ["Plus tard", "Optionnel"]

# pop sans index : supprime le dernier
derniere = taches.pop()  # derniere = "Optionnel"
# Résultat : ["Plus tard"]

# clear : vide tout
taches.clear()
# Résultat : []
```

Différence remove vs pop :

```
taches = ["A", "B", "C"]

# remove : on doit connaître la valeur
taches.remove("B")  # ✓ Supprime "B"

# pop : on utilise l'index (position)
taches.pop(1)  # ✓ Supprime l'élément à la position 1 (qui était "B")
```

Résultat attendu

```
--- MENU ---
1. Afficher les tâches
2. Ajouter une tâche
3. Supprimer une tâche
4. Quitter

Choix : 1

=== MES TÂCHES ===
1. Apprendre Python
2. Faire les courses
3. Réviser le code

Choix : 3

=== MES TÂCHES ===
1. Apprendre Python
2. Faire les courses
3. Réviser le code

Numéro de la tâche à supprimer : 2
✓ Tâche 'Faire les courses' supprimée !
```

ÉTAPE 4 : Modifier des éléments

Objectif

Pouvoir modifier une tâche existante sans la supprimer et recréer.

Le code (Version avec modification)

```
def modifier_tache():
    """Modifie une tâche existante"""
    if not taches:
        print("\n☒ Aucune tâche à modifier !")
        return

    afficher_taches()

    try:
        numero = int(input("\nNuméro de la tâche à modifier : "))

        if 1 <= numero <= len(taches):
            ancienne = taches[numero - 1]
            nouvelle = input(f"Nouvelle description (actuelle: '{ancienne}') : ")

            # Modification directe par index
            taches[numero - 1] = nouvelle
            print(f"✓ Tâche modifiée : '{ancienne}' → '{nouvelle}'")
        else:
            print("\n☒ Numéro invalide !")
    except ValueError:
        print("\n☒ Entre un numéro valide !")
```

Explication ligne par ligne

Ligne 13 :

```
ancienne = taches[numero - 1]
```

→ **Accès par index pour lire.**

Ligne 16 :

```
taches[numero - 1] = nouvelle
```

→ **Accès par index pour écrire (modifier).**

Ce qui se passe :

- `liste[index]` = Accède à l'élément à la position `index`
- On peut lire (`x = liste[0]`)
- On peut écrire (`liste[0] = "Nouveau"`)

Opérations sur les listes

Opération	Syntaxe	Exemple
Lire	`x = liste[i]`	`tache = taches[0]`
Écrire	`liste[i] = x`	`taches[0] = "Nouveau"`
Longueur	`len(liste)`	`nb = len(taches)`
Vérifier présence	`x in liste`	`if "Test" in taches:`
Index d'un élément	`liste.index(x)`	`pos = taches.index("Test")`

Résultat attendu

```
=== MES TÂCHES ===
1. Apprendre Python
2. Réviser le code

Numéro de la tâche à modifier : 1
Nouvelle description (actuelle: 'Apprendre Python') : Maîtriser Python
✓ Tâche modifiée : 'Apprendre Python' → 'Maîtriser Python'
```

PARTIE 2 : Les fichiers (45min)

ÉTAPE 5 : Écrire dans un fichier

Objectif

Sauvegarder les tâches dans un fichier texte.

Le code (Version basique)

```
# Sauvegarde dans un fichier texte

def sauvegarder_txt():
    """Sauvegarde les tâches dans un fichier texte"""
    with open("taches.txt", "w", encoding="utf-8") as fichier:
        for tache in taches:
            fichier.write(tache + "\n")
    print("✓ Tâches sauvegardées dans 'taches.txt' !")
```

Explication ligne par ligne

Ligne 5 :

```
with open("taches.txt", "w", encoding="utf-8") as fichier:
```

→ **Ouverture d'un fichier en mode écriture.**

Décomposition :

- `open("taches.txt", "w")` = Ouvre le fichier en mode **write** (écriture)
- `encoding="utf-8"` = Supporte les accents et caractères spéciaux
- `as fichier` = Donne un nom de variable pour manipuler le fichier
- `with` = Ferme automatiquement le fichier à la fin

Ligne 7 :

```
fichier.write(tache + "\n")
```

→ **Écriture dans le fichier.**

Ce qui se passe :

- `.write(texte)` écrit le texte dans le fichier
- `+ "\n"` ajoute un retour à la ligne (sinon tout serait sur une ligne)

Les modes d'ouverture

Mode	Signification	Action
`"r"`	Read (lecture)	Lit le fichier (erreur si inexistant)
`"w"`	Write (écriture)	Crée ou **écrase** le fichier
`"a"`	Append (ajout)	Ajoute à la fin (crée si inexistant)
`"r+"`	Read + Write	Lecture ET écriture

Attention avec "w" :

```
# Si taches.txt existe déjà, il sera ÉCRASÉ !
with open("taches.txt", "w") as f:
    f.write("Nouveau contenu")
# L'ancien contenu est perdu !
```

Exemple complet de sauvegarde

```
taches = ["Apprendre Python", "Faire les courses", "Réviser"]

# Sauvegarder
with open("taches.txt", "w", encoding="utf-8") as fichier:
    for tache in taches:
        fichier.write(tache + "\n")

print("✓ Sauvegardé !")
```

Contenu de taches.txt :

```
Apprendre Python
Faire les courses
Réviser
```

ÉTAPE 6 : Lire depuis un fichier

Objectif

Charger les tâches sauvegardées au démarrage du programme.

Le code (Version basique)

```
def charger_txt():
    """Charge les tâches depuis un fichier texte"""
    try:
        with open("taches.txt", "r", encoding="utf-8") as fichier:
            # Lire toutes les lignes
            lignes = fichier.readlines()

            # Nettoyer et ajouter à la liste
            for ligne in lignes:
                tache = ligne.strip()  # Enlève \n et espaces
                if tache:  # Ignore les lignes vides
                    taches.append(tache)

        print(f"✓ {len(taches)} tâche(s) chargée(s) !")
    except FileNotFoundError:
        print("→ Aucun fichier de sauvegarde trouvé. Nouveau départ !")
```

Explication ligne par ligne

Ligne 6 :

```
lignes = fichier.readlines()
```

→ **Méthode .readlines() !**

Ce qui se passe :

- Lit TOUT le fichier
- Retourne une **liste** où chaque élément = une ligne
- Exemple : `["Ligne 1\n", "Ligne 2\n", "Ligne 3\n"]`

Ligne 10 :

```
tache = ligne.strip()
```

→ **Méthode .strip() !**

Ce qui se passe :

- Enlève les espaces au début et à la fin
- Enlève aussi les `\n` (retours à la ligne)
- `"  Tâche  \n"` devient `"Tâche"`

Ligne 15 :

```
except FileNotFoundError:
```

→ **Gestion d'erreur si le fichier n'existe pas.**

Les méthodes de lecture

Méthode	Retourne	Utilité
`.read()`	String (tout le fichier)	Petit fichier, texte brut
`.readline()`	String (une ligne)	Lire ligne par ligne
`.readlines()`	Liste de strings	Toutes les lignes d'un coup

Exemples :

```python
# Fichier taches.txt :
# Ligne 1
# Ligne 2
# Ligne 3

# read() : tout en une string
with open("taches.txt", "r") as f:
    contenu = f.read()
# contenu = "Ligne 1\nLigne 2\nLigne 3\n"

# readlines() : liste de lignes
with open("taches.txt", "r") as f:
    lignes = f.readlines()
# lignes = ["Ligne 1\n", "Ligne 2\n", "Ligne 3\n"]

# readline() : une ligne à la fois
with open("taches.txt", "r") as f:
    ligne1 = f.readline()  # "Ligne 1\n"
    ligne2 = f.readline()  # "Ligne 2\n"
```

Programme avec chargement au démarrage

```python
taches = []

# Charger au démarrage
charger_txt()

# Menu principal
while True:
    print("\n--- MENU ---")
    # ... menu ...

    if choix == "4":  # Quitter
        sauvegarder_txt()
        print("\nÀ bientôt !")
        break
```

ÉTAPE 7 : Le format JSON

Objectif

Utiliser JSON pour sauvegarder des structures plus complexes (tâches avec statut, priorité, etc.).

C'est quoi JSON ?

JSON = JavaScript Object Notation

Un format **texte** pour stocker des données structurées :

```
{
  "nom": "Pascal",
  "age": 25,
  "taches": ["Python", "Sport"],
  "actif": true
}
```

Avantages :

- ✓ Lisible par les humains
- ✓ Supporte listes, dictionnaires, nombres, booléens
- ✓ Standard universel (web, APIs, bases de données)

Le code (Version JSON simple)

```python
import json

def sauvegarder_json():
    """Sauvegarde les tâches en JSON"""
    with open("taches.json", "w", encoding="utf-8") as fichier:
        json.dump(taches, fichier, indent=2, ensure_ascii=False)
    print("✓ Tâches sauvegardées !")

def charger_json():
    """Charge les tâches depuis JSON"""
    global taches
    try:
        with open("taches.json", "r", encoding="utf-8") as fichier:
            taches = json.load(fichier)
        print(f"✓ {len(taches)} tâche(s) chargée(s) !")
    except FileNotFoundError:
        print("→ Aucun fichier trouvé. Nouveau départ !")
```

Explication ligne par ligne

Ligne 6 :

```python
json.dump(taches, fichier, indent=2, ensure_ascii=False)
```

→ **Sauvegarder en JSON.**

Paramètres :

- `taches` = Les données à sauvegarder
- `fichier` = Où sauvegarder
- `indent=2` = Formate joliment (avec indentation)
- `ensure_ascii=False` = Garde les accents

Ligne 14 :

```python
taches = json.load(fichier)
```

→ **Charger depuis JSON.**

Ce qui se passe :

- Lit le fichier JSON
- Transforme en objet Python (liste, dict, etc.)

JSON avec structure complexe

```python
# Tâches avec statut et priorité
taches = [
    {
        "titre": "Apprendre Python",
        "terminee": False,
        "priorite": "haute"
    },
    {
        "titre": "Faire les courses",
        "terminee": True,
        "priorite": "moyenne"
    }
]

# Sauvegarder
with open("taches.json", "w", encoding="utf-8") as f:
    json.dump(taches, f, indent=2, ensure_ascii=False)
```

Fichier taches.json généré :

```json
[
  {
    "titre": "Apprendre Python",
    "terminee": false,
    "priorite": "haute"
  },
  {
    "titre": "Faire les courses",
    "terminee": true,
    "priorite": "moyenne"
  }
]
```

Conversions Python ↔ JSON

Python	JSON
dict	object
list	array
str	string
int, float	number
True	true
False	false
None	null

PARTIE 3 : TODO List complète (30min)

ÉTAPE 8 : Version console

Objectif

Créer une TODO list console complète avec toutes les fonctionnalités.

Le code (Version complète)

```python
# ==========================================
# TODO LIST COMPLÈTE - VERSION CONSOLE
# Auteur : C Premiers Pas
# Version : 1.0 (vCODE #6)
# ==========================================

import json
import os

# Liste des tâches
taches = []

def afficher_menu():
    """Affiche le menu principal"""
    print("\n" + "="*40)
    print("     TODO LIST - GESTION DE TÂCHES")
    print("="*40)
    print("1. Afficher toutes les tâches")
    print("2. Ajouter une tâche")
    print("3. Marquer comme terminée")
    print("4. Supprimer une tâche")
    print("5. Modifier une tâche")
    print("6. Sauvegarder")
    print("7. Quitter")
    print("="*40)

def afficher_taches():
    """Affiche toutes les tâches"""
    if not taches:
        print("\n✓ Aucune tâche pour le moment.")
        return

    print("\n" + "-"*40)
    print("          LISTE DES TÂCHES")
    print("-"*40)

    for i, tache in enumerate(taches, 1):
        statut = "✓" if tache["terminee"] else "o"
        titre = tache["titre"]

        # Affichage avec statut
```

```
        if tache["terminee"]:
            print(f"{statut} {i}. [{titre}] (Terminée)")
        else:
            print(f"{statut} {i}. {titre}")

    print("-"*40)
    print(f"Total : {len(taches)} tâche(s)")

def ajouter_tache():
    """Ajoute une nouvelle tâche"""
    titre = input("\nTitre de la tâche : ").strip()

    if not titre:
        print("☒ Le titre ne peut pas être vide !")
        return

    nouvelle_tache = {
        "titre": titre,
        "terminee": False
    }

    taches.append(nouvelle_tache)
    print(f"✓ Tâche '{titre}' ajoutée !")

def marquer_terminee():
    """Marque une tâche comme terminée"""
    if not taches:
        print("\n☒ Aucune tâche à marquer !")
        return

    afficher_taches()

    try:
        numero = int(input("\nNuméro de la tâche à marquer : "))

        if 1 <= numero <= len(taches):
            taches[numero - 1]["terminee"] = True
            titre = taches[numero - 1]["titre"]
            print(f"✓ Tâche '{titre}' marquée comme terminée !")
        else:
            print("☒ Numéro invalide !")
    except ValueError:
        print("☒ Entre un numéro valide !")
```

```python
def supprimer_tache():
    """Supprime une tâche"""
    if not taches:
        print("\n☒ Aucune tâche à supprimer !")
        return

    afficher_taches()

    try:
        numero = int(input("\nNuméro de la tâche à supprimer : "))

        if 1 <= numero <= len(taches):
            tache_supprimee = taches.pop(numero - 1)
            print(f"✓ Tâche '{tache_supprimee['titre']}' supprimée !")
        else:
            print("☒ Numéro invalide !")
    except ValueError:
        print("☒ Entre un numéro valide !")

def modifier_tache():
    """Modifie une tâche existante"""
    if not taches:
        print("\n☒ Aucune tâche à modifier !")
        return

    afficher_taches()

    try:
        numero = int(input("\nNuméro de la tâche à modifier : "))

        if 1 <= numero <= len(taches):
            ancienne = taches[numero - 1]["titre"]
            nouvelle = input(f"Nouveau titre ('{ancienne}') : ").strip()

            if nouvelle:
                taches[numero - 1]["titre"] = nouvelle
                print(f"✓ Tâche modifiée : '{ancienne}' → '{nouvelle}'")
            else:
                print("☒ Le titre ne peut pas être vide !")
        else:
            print("☒ Numéro invalide !")
    except ValueError:
```

```python
        print("⊠ Entre un numéro valide !")

def sauvegarder():
    """Sauvegarde les tâches dans un fichier JSON"""
    try:
        with open("taches.json", "w", encoding="utf-8") as fichier:
            json.dump(taches, fichier, indent=2, ensure_ascii=False)
        print("✓ Tâches sauvegardées dans 'taches.json' !")
    except Exception as e:
        print(f"⊠ Erreur lors de la sauvegarde : {e}")

def charger():
    """Charge les tâches depuis le fichier JSON"""
    global taches

    if not os.path.exists("taches.json"):
        print("→ Aucun fichier de sauvegarde. Nouveau départ !")
        return

    try:
        with open("taches.json", "r", encoding="utf-8") as fichier:
            taches = json.load(fichier)
        print(f"✓ {len(taches)} tâche(s) chargée(s) !")
    except Exception as e:
        print(f"⊠ Erreur lors du chargement : {e}")

# === PROGRAMME PRINCIPAL ===
def main():
    """Fonction principale"""
    print("\n● BIENVENUE DANS TODO LIST ●")
    print("-" * 40)

    # Charger les tâches au démarrage
    charger()

    # Boucle principale
    while True:
        afficher_menu()
        choix = input("\nTon choix : ").strip()

        if choix == "1":
            afficher_taches()
        elif choix == "2":
```

```
            ajouter_tache()
        elif choix == "3":
            marquer_terminee()
        elif choix == "4":
            supprimer_tache()
        elif choix == "5":
            modifier_tache()
        elif choix == "6":
            sauvegarder()
        elif choix == "7":
            # Sauvegarder avant de quitter
            sauvegarder()
            print("\n● À bientôt ! ●\n")
            break
        else:
            print("\n⊠ Choix invalide !")

# Point d'entrée
if __name__ == "__main__":
    main()
```

Points clés du code

Structure des tâches :

```
tache = {
    "titre": "Apprendre Python",
    "terminee": False
}
```

Sauvegarde automatique :

- Au démarrage : `charger()`
- En quittant : `sauvegarder()`
- Manuellement : Option 6

Gestion d'erreurs :

- `try/except` pour les entrées utilisateur
- Vérification des listes vides
- Validation des numéros

ÉTAPE 9 : Sauvegarde automatique

Objectif

Sauvegarder automatiquement après chaque modification.

Code amélioré

```python
def auto_save(func):
    """Décorateur pour sauvegarder automatiquement"""
    def wrapper(*args, **kwargs):
        result = func(*args, **kwargs)
        sauvegarder()
        return result
    return wrapper

@auto_save
def ajouter_tache():
    # ... code ...
    pass

@auto_save
def supprimer_tache():
    # ... code ...
    pass
```

Ou version simple :

```python
def ajouter_tache():
    # ... ajout de la tâche ...
    taches.append(nouvelle_tache)
    print(f"✓ Tâche '{titre}' ajoutée !")

    # Sauvegarde automatique
    sauvegarder()
```

ÉTAPE 10 : Version finale

Le code de l'étape 8 est déjà la version finale ! Il contient :

✓ CRUD complet (Create, Read, Update, Delete)
✓ Sauvegarde/Chargement JSON
✓ Gestion d'erreurs robuste
✓ Interface console claire
✓ Code bien organisé en fonctions

Aller plus loin

1. Priorités et dates

```
import datetime

def ajouter_tache_avancee():
    titre = input("Titre : ")
    priorite = input("Priorité (haute/moyenne/basse) : ")
    date_limite = input("Date limite (AAAA-MM-JJ) : ")

    tache = {
        "titre": titre,
        "terminee": False,
        "priorite": priorite,
        "date_limite": date_limite,
        "date_creation": str(datetime.date.today())
    }

    taches.append(tache)
```

2. Tri et filtrage

```python
def trier_par_priorite():
    """Trie les tâches par priorité"""
    priorites = {"haute": 1, "moyenne": 2, "basse": 3}
    taches.sort(key=lambda t: priorites.get(t.get("priorite", "moyenne"), 2))

def afficher_non_terminees():
    """Affiche seulement les tâches non terminées"""
    non_terminees = [t for t in taches if not t["terminee"]]
    for i, tache in enumerate(non_terminees, 1):
        print(f"{i}. {tache['titre']}")
```

3. Catégories

```python
def ajouter_avec_categorie():
    titre = input("Titre : ")
    categorie = input("Catégorie (travail/perso/urgent) : ")

    tache = {
        "titre": titre,
        "categorie": categorie,
        "terminee": False
    }

    taches.append(tache)

def filtrer_par_categorie(categorie):
    return [t for t in taches if t.get("categorie") == categorie]
```

4. Export CSV

```python
import csv

def exporter_csv():
    """Exporte les tâches en CSV"""
    with open("taches.csv", "w", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=["titre", "terminee"])
        writer.writeheader()
        writer.writerows(taches)
    print("✓ Exporté vers taches.csv")
```

5. Version Tkinter

```python
import tkinter as tk
from tkinter import messagebox

class TodoApp:
    def __init__(self):
        self.fenetre = tk.Tk()
        self.fenetre.title("TODO List")

        # Zone de saisie
        self.entry = tk.Entry(self.fenetre, width=40)
        self.entry.pack(pady=10)

        # Bouton ajouter
        tk.Button(self.fenetre, text="Ajouter", command=self.ajouter).pack()

        # Liste des tâches
        self.listbox = tk.Listbox(self.fenetre, width=50, height=15)
        self.listbox.pack(pady=10)

        # Bouton supprimer
        tk.Button(self.fenetre, text="Supprimer", command=self.supprimer).pack()

        self.charger()

    def ajouter(self):
        tache = self.entry.get()
        if tache:
            self.listbox.insert(tk.END, tache)
            self.entry.delete(0, tk.END)
            self.sauvegarder()

    def supprimer(self):
        selection = self.listbox.curselection()
        if selection:
            self.listbox.delete(selection)
            self.sauvegarder()

    def sauvegarder(self):
        taches = list(self.listbox.get(0, tk.END))
        with open("taches.json", "w") as f:
            json.dump(taches, f)
```

```python
    def charger(self):
        try:
            with open("taches.json", "r") as f:
                taches = json.load(f)
                for tache in taches:
                    self.listbox.insert(tk.END, tache)
        except:
            pass

    def lancer(self):
        self.fenetre.mainloop()

if __name__ == "__main__":
    app = TodoApp()
    app.lancer()
```

Récapitulatif

Ce que tu as appris

Concept	Maîtrise	Application
Listes avancées	✓	Stocker des collections
append, remove, pop	✓	Manipuler des listes
Fichiers texte	✓	Sauvegarder des données
Format JSON	✓	Structures complexes
CRUD	✓	Create, Read, Update, Delete
Persistance	✓	Données qui survivent

Progression

vCODE #3 (Calculatrice) :

- Variables, fonctions, opérations

vCODE #4 (Jeu) :

- Random, boucles, comparaisons

vCODE #5 (Tkinter) :

- Interfaces graphiques, événements

vCODE #6 (TODO List) : ✓ NOUVEAU

- Listes avancées
- Fichiers et JSON
- CRUD complet
- Persistance de données

Ce que tu peux créer maintenant

✓ **Gestionnaires** : Notes, contacts, inventaires
✓ **Applications de suivi** : Habitudes, budgets, temps
✓ **Bases de données simples** : Collections, catalogues
✓ **Systèmes de gestion** : Stocks, commandes, réservations

Conclusion

Le chemin parcouru

Il y a 2 heures, tu ne savais peut-être pas manipuler des fichiers. Maintenant :

✓ Tu sais créer et manipuler des listes
✓ Tu maîtrises la sauvegarde/chargement de fichiers
✓ Tu comprends le format JSON
✓ Tu as créé une application complète avec persistance
✓ Tu peux gérer des données comme un pro

Ce que tu as vraiment appris

Les données persistent : Fichiers, JSON, CSV
Le CRUD : Base de toute application

L'organisation : Fonctions, structure, clarté
La robustesse : Gestion d'erreurs, validation

Prochaine étape

Tu as maintenant les bases pour créer **n'importe quelle application de gestion de données** !

Challenge : Crée ta propre application :

- Gestionnaire de notes
- Tracker d'habitudes
- Catalogue de films/livres
- Gestionnaire de dépenses
- Journal intime numérique

Tu as toutes les compétences !

> *"Les données sont le pétrole du 21ème siècle. Aujourd'hui, tu as appris à les stocker, les gérer et les faire persister."*
>
> **— C Premiers Pas**

Document réalisé par C Premiers Pas

📧 **Contact :** NOUS CONTACTER

🌐 **Rejoignez notre communauté**

LINKEDIN
WHATSAPP
DISCORD

© 2026 C Premiers Pas - Tous droits réservés